

Data and Artificial Intelligence

Cyber Shujaa Program

Week 12 Assignment – Generative AI and Retrieval-Augmented Generation (RAG)

Student Name: Hope Njoki Nyambura

Student ID: cs-eh03-25100

Table of Contents

1. Introduction	1
2. Data Description	2
3. Methods and Model Description	2
3.1 PDF Loading and Chunking.....	2
3.2 Embeddings and Vector Store	2
3.3 Generative Model (Flan-T5).....	3
3.4 Retrieval-Augmented Generation (RAG)	4
4. Results	4
4.1 RAG Outputs	4
4.2 Saved Outputs.....	5
5. Key Concepts.....	5
6. Conclusion	5
7. Google Colab Link.....	5

1. Introduction

This assignment demonstrates the application of Generative AI and Retrieval-Augmented Generation (RAG) to answer questions using context from a PDF document. The main objective was to implement a RAG pipeline that retrieves relevant text chunks and generates context-aware answers using a generative model (Flan-T5).

2. Data Description

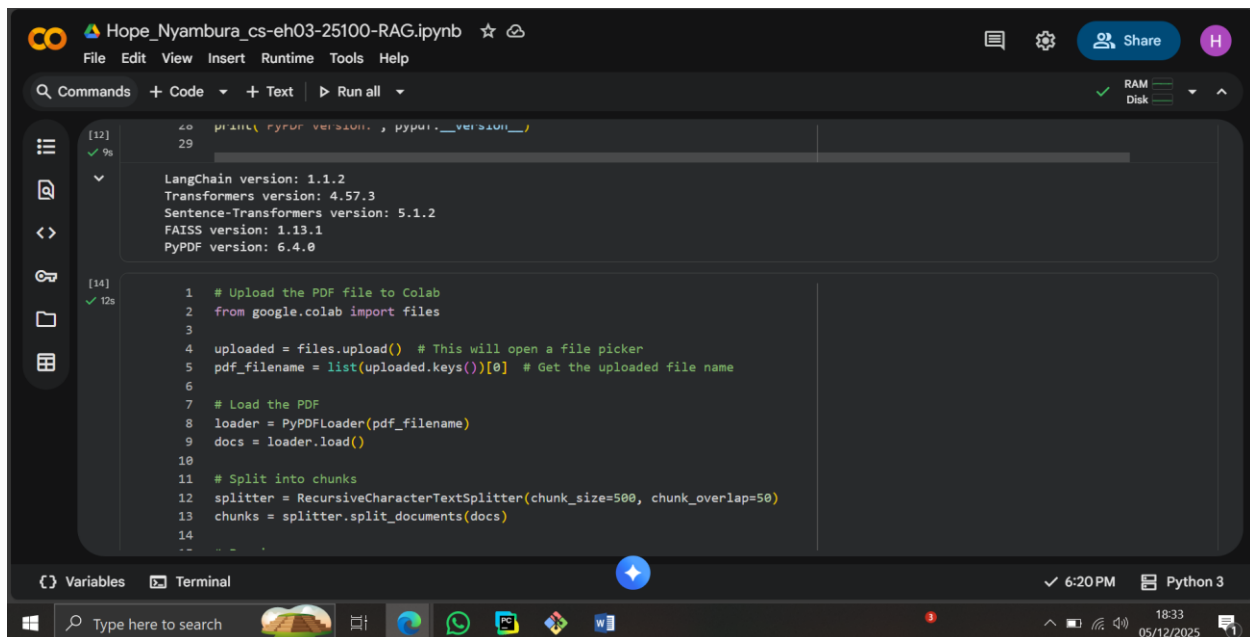
The input data consisted of a PDF document titled document.pdf. The document included content on Generative AI and RAG concepts. For processing:

- The PDF was split into smaller chunks of 500 characters with 50-character overlap to preserve context.
- A total of 6 chunks were generated.

3. Methods and Model Description

3.1 PDF Loading and Chunking

- Uploaded the PDF to Colab.
- Used PyPDFLoader to read the document.
- Split text into chunks using RecursiveCharacterTextSplitter.



The screenshot shows a Google Colab notebook titled 'Hope_Nyambura_cs-eh03-25100-RAG.ipynb'. The notebook has two code cells. The first cell, labeled [12], contains code to print the versions of various libraries: LangChain, Transformers, Sentence-Transformers, FAISS, and PyPDF. The second cell, labeled [14], contains code to upload a PDF file, load it using PyPDFLoader, and split it into chunks using RecursiveCharacterTextSplitter with a chunk size of 500 and a chunk overlap of 50. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running all cells, and a sidebar with icons for variables, terminal, and other tools. The bottom status bar shows the time as 6:20 PM and the Python version as Python 3.

```
[12] 40 print('PyPDF version: ', pyppdf.__version__)
      29
      LangChain version: 1.1.2
      Transformers version: 4.57.3
      Sentence-Transformers version: 5.1.2
      FAISS version: 1.13.1
      PyPDF version: 6.4.0

[14] 1 # Upload the PDF file to Colab
      2 from google.colab import files
      3
      4 uploaded = files.upload() # This will open a file picker
      5 pdf_filename = list(uploaded.keys())[0] # Get the uploaded file name
      6
      7 # Load the PDF
      8 loader = PyPDFLoader(pdf_filename)
      9 docs = loader.load()
      10
      11 # Split into chunks
      12 splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
      13 chunks = splitter.split_documents(docs)
      14
```

3.2 Embeddings and Vector Store

- Generated embeddings for each chunk using HuggingFace Sentence-Transformers (all-MiniLM-L6-v2).
- Built a FAISS vector store for efficient similarity search.
- Verified that the retriever was ready.

The screenshot shows a Jupyter Notebook titled 'Hope_Nyambura_cs-eh03-25100-RAG.ipynb'. The code in the cell is as follows:

```
1 # Create embeddings for each chunk and build the FAISS vector store
2 embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
3 vectorstore = FAISS.from_documents(chunks, embeddings)
4 retriever = vectorstore.as_retriever()
5
6 print("Vector store created. Retriever is ready.")
7
```

The output of the cell shows a warning from LangChain about the deprecation of `HuggingFaceEmbeddings` and a `UserWarning` from the `huggingface_hub` library stating that the secret `'HF_TOKEN'` does not exist. Below the warnings, progress bars indicate the download status of `modules.json` (100% at 349/349 KB/s) and `config_sentence_transformers.json` (100% at 116/116 KB/s). The `README.md` file is also shown as downloaded (10.5k? at 723k/s).

3.3 Generative Model (Flan-T5)

- Used Flan-T5 Large via HuggingFace pipeline for text generation.
- Configured for text2text generation with sampling parameters (temperature=0.9, top_k=50, top_p=0.9).

The screenshot shows the same Jupyter Notebook interface, now displaying the output of the previous cell: "Vector store created. Retriever is ready." The code in the next cell is as follows:

```
1 # Query function using FAISS directly (no LangChain chains)
2 def query_rag(question, top_k=3):
3     # Get top-k similar documents
4     results = vectorstore.similarity_search(question, k=top_k)
5
6     # Combine content from retrieved chunks
7     context = "\n".join([doc.page_content for doc in results])
8
9     # Prepare the prompt
10    prompt = f"Answer the question using only the context:\n\nContext:\n{context}\n\nQuestion: {question}\n\nAnswer:"
11
12    # Generate answer with Flan-T5 pipeline
13    response = flan_pipeline(
14        prompt,
15        max_new_tokens=200,
16        temperature=0.9,
17        top_k=50,
18        top_p=0.9,
19        do_sample=True
20    )
```

3.4 Retrieval-Augmented Generation (RAG)

- Implemented query_rag function to:
 1. Retrieve top-k relevant chunks from FAISS using similarity_search.
 2. Combine retrieved chunks as context.
 3. Generate answers with the Flan-T5 model.

4. Results

4.1 RAG Outputs

Example outputs from queries:

- **Query:** “Summarize the key points of this document in 200 words.”
Answer:

Retrieval-Augmented Generation (RAG) is an AI framework that combines two powerful components:

1. A retrieval system that fetches relevant documents.
 2. A generative model that uses the retrieved information to produce accurate responses.
- RAG helps AI overcome limitations such as hallucinations and outdated knowledge by grounding responses in real documents.

- **Query:** “What are the main steps in the RAG pipeline described in this document?”
Answer:

1. A user asks a question.
2. The system retrieves relevant text chunks from a knowledge base.
3. The retrieved information is combined with the question.
4. The generative model produces an answer based on the retrieved context.

- **Query:** “Explain how embeddings are created and used for retrieval in this assignment.”
Answer:

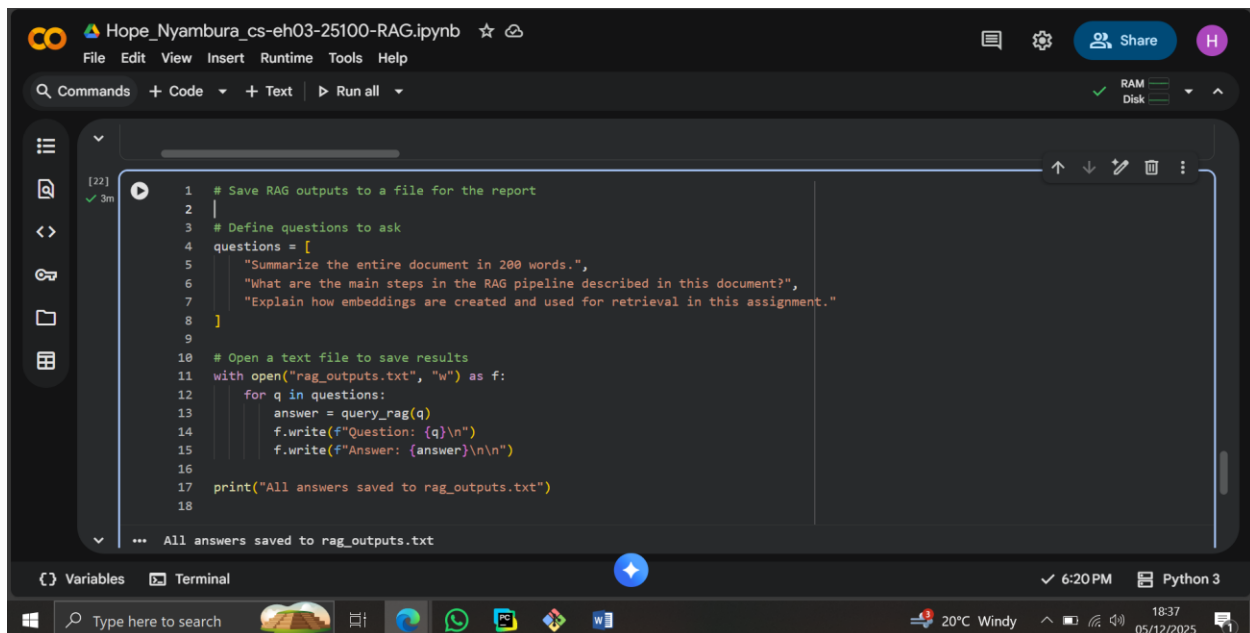
Embeddings were generated for each document chunk using Sentence-Transformers (all-MiniLM-L6-v2).

These embeddings represent the semantic meaning of each chunk in vector form.

The FAISS vector store uses these embeddings to retrieve the most relevant chunks for a given question.

4.2 Saved Outputs

- All outputs were saved in rag_outputs.txt for submission and report purposes.



The screenshot shows a Google Colab notebook titled 'Hope_Nyambura_cs-eh03-25100-RAG.ipynb'. The code is as follows:

```
1 # Save RAG outputs to a file for the report
2 |
3 # Define questions to ask
4 questions = [
5     "Summarize the entire document in 200 words.",
6     "What are the main steps in the RAG pipeline described in this document?",
7     "Explain how embeddings are created and used for retrieval in this assignment."
8 ]
9
10 # Open a text file to save results
11 with open("rag_outputs.txt", "w") as f:
12     for q in questions:
13         answer = query_rag(q)
14         f.write(f"Question: {q}\n")
15         f.write(f"Answer: {answer}\n\n")
16
17 print("All answers saved to rag_outputs.txt")
18
```

The output of the code is displayed at the bottom: 'All answers saved to rag_outputs.txt'.

5. Key Concepts

1. **Generative AI** – Systems that create new content, e.g., text or images.
2. **RAG** – Combines retrieval and generative models for context-aware answers.
3. **FAISS** – Efficient similarity search library for vector embeddings.
4. **Sentence Embeddings** – Vector representations of text chunks capturing semantic meaning.
5. **Flan-T5** – Generative transformer model used for producing answers from context.

6. Conclusion

- Successfully implemented a RAG pipeline in Python using LangChain, FAISS, and Flan-T5.
- Demonstrated retrieval of relevant document chunks and generation of context-aware answers.
- Learned how embeddings and vector stores improve generative model accuracy.
- Gained hands-on experience with PDF processing, vector databases, and generative AI pipelines.

7. Google Colab Link

[<https://colab.research.google.com/drive/19ThxzMG2kw3nA27oa16u21g4BgBe9Q3j?usp=sharing>]